

Cheat Sheet: Conceitos C++

MÓDULO 01 • 42 PORTO

ESTUDANTE

Leonardo La Rocque

ex00 Stack vs Heap (Alocação de Memória)

Stack (Estática): Instanciar um objeto diretamente (ex: `Zombie z;`). A memória é alocada e libertada automaticamente quando o objeto sai do seu contexto (scope). Ideal para operações rápidas e locais.

Heap (Dinâmica): Instanciar usando `new` devolve um ponteiro (ex: `Zombie* z = new Zombie();`). A memória sobrevive após o fim da função e obriga o programador a geri-la rigorosamente usando o comando `delete` para evitar *Memory Leaks*.

ex01 Arrays Dinâmicos

A alocação de múltiplos objetos contíguos na Heap é feita com `new[]`. O C++ invoca o construtor por omissão para cada objeto do array. Consequentemente, para libertar esta memória de forma correta e destruir todos os objetos, é estritamente necessário usar o operador de vetor `delete[]`, em vez do `delete` normal.

ex02 Ponteiros (*) vs Referências (&)

Ponteiros: Variáveis que guardam endereços de memória. Podem ser inicializadas a `NULL`, podem mudar de alvo ao longo do programa e necessitam de desreferenciação explícita.

Referências: São "apelidos" (alias) para variáveis já existentes. Têm restrições severas que aumentam a segurança: devem ser obrigatoriamente inicializadas na sua criação, não podem apontar para `NULL` e nunca podem mudar de alvo. Partilham os mesmos operadores da variável original (usa-se `.` em vez de `->`).

ex03 Design de Classes e Composição

A escolha entre usar Ponteiros ou Referências como atributos privados dita o comportamento da classe. Se um Humano **tem obrigatoriamente** de ter uma Arma desde que nasce, a Arma deve ser uma Referência `Weapon&`. Se o Humano pode estar desarmado, a Arma deve ser um Pontoeiro `Weapon*` inicializado a `NULL`.

ex04 File Streams e Manipulação de Strings

A biblioteca `<fstream>` substitui o C clássico através de objetos `std::ifstream` (leitura) e `std::ofstream` (escrita). O exercício explorou a manipulação avançada de `std::string`, combinando os métodos `.find()`, `.erase()` e `.insert()` para substituir texto de forma orgânica. Aprendemos a usar a constante `std::string::npos` para verificar falhas na pesquisa.

ex05 Ponteiros para Funções Membro

Para evitar a complexidade ciclomática de ramificações `if/else`, o C++ permite armazenar referências de métodos em arrays de ponteiros. A sintaxe de declaração é `void (Class::*ptr[])(void)`, sendo posteriormente invocada associando o objeto atual através de `(this->*ptr[i])()`. É uma ferramenta vital para o *routing* de comandos.

ex06 A Instrução Switch e o Fall-Through

A instrução `switch` está restrita a comparar tipos primitivos inteiros, caracteres ou enumerações, rejeitando categoricamente objetos como a `std::string`. A omissão intencional da keyword `break;` gera um efeito de **Fall-through**, forçando o programa a continuar a execução pelos casos (cases) inferiores em cascata. O uso de comentários como `/* fall through */` suprime alertas de segurança do compilador.